

Reflection

Kevin Rebakure

Grid was easier for my part (Level 3) because the named areas made it easier to see where each section should go, especially when moving from tablet to desktop. Flexbox worked well for the one-direction sections like the header and navigation, but Grid felt clearer for positioning the three main page regions. In grid.css, I used grid-template-areas: “main main” “left right”; for tablet then “left main right” with 1fr 2.5fr 1fr on desktop. Getting the tablet transition and sidebar resising took longer because the areas, columns, and breakpoints all had to work together.

Lincoln

Grid was definitely easier for this 70/30 layout because `grid-template-columns: 7fr 3fr;` handled the proportions directly without requiring extra calculations. In Flexbox, achieving the split was trickier. I had to use `flex-basis: 68%;` and `flex-basis: 30%;` with a `gap: 2%;` on `.flex-layout`. If the `flex-basis` values plus the gap exceed 100%, the sidebar wraps to the next line unexpectedly. What took longer than expected was making sure all the breakpoints (Mobile, Tablet, and Desktop) worked identically across both layouts without adding unnecessary or repeated media queries.

Rhoda

I started by creating the basic structure with a header, navigation links, a main content area, a hero image, an article section, and a footer. In flex.css, I used flexbox to arrange the elements and keep the layout neat. Also added media queries so the page works on desktop, tablets, and phones. The main challenge was making the design responsive without ruining the desktop layout, but adjusting the spacing, sizes, and direction helped solve it

Mitchell

For Level 4, I built the HTML, the Flexbox, and the sidebar toggle script. Flexbox felt easier and more intuitive to me, especially for one-direction items like the header, nav, and footer columns, and flex-wrap with container queries let the cards reflow when a sidebar collapsed. The media queries were the most difficult part, since each breakpoint needed extra rules to handle the toggle for the sidebar and force the content back into view: `.sidebar-right .sidebar-toggle { display: none; } .sidebar-right .sidebar-toggle + .sidebar-content { display: block; }` My flex.css came to 283 lines against 404 for grid.css, though Grid’s mosaic cards added extra features. I prefer flexbox for components, but grid for the overall page structure.

Mufaro

I did Level 4, I built the grid html, the Grid and its responsiveness. CSS Grid was easier for me to implement because the .layout had multiple sections which allowed me to control the layout. At tablet size, I used grid-template-areas: “left main” “right right” so the sidebar moved below the main content without changing the HTML. Collapsing a sidebar was also simpler because .layout:has(...) only changes the - - left-sidebar or - - right-sidebar to - - sidebar-collapsed, instead of adjusting each sidebar’s flex-basis like the Flexbox version. Responsiveness took longer than expected because I had to test each breakpoint and ensure the Grid version still

matched the Flexbox layout visually.